

FinTechAI Scanner Pipeline

A plain-language, function-by-function walkthrough of how LaunchPad, Alpha Zone, and the other scanners actually run — from the cron trigger down to the JSON the frontend renders. Every file and function name below is exactly what's in the code.

Contents

- 1. The big picture — three layers
- 2. What actually starts a scan (3 triggers)
- 3. Step 1 — the data pipeline (run_daily_scan)
- 4. Step 2 — inside ONE strategy (LaunchPad, function by function)
- 5. Step 3 — Alpha Zone (what's different)
- 6. Step 4 — how the frontend gets the results
- 7. Confidence & scoring — the exact formulas
- 8. Where things can go quietly wrong (diagnostic map)

1. The big picture — three layers

Think of the scanner system as three layers stacked on top of each other. Nothing skips a layer.

Layer	What it does	Key files
1. Data pipeline	Downloads/refreshes raw OHLCV candles for ~4,200 NSE stocks and serializes them down to a big in-memory dictionary .	services/ohlc/download/schedulers/daily_refresh.py
2. Engines	Pure calculation code: indicators (EMA/RSI/ATR), pattern detection (FVG, indicators), and each proprietary strategy's	engines/patterns/ engines/strategies/ engines/ranking/
3. API + cache	Persists each strategy's qualifying stocks into its own Mongo collection and serves it to the frontend with filters/pagination .	api/servers.py

There are currently **six scanners** sharing this pipeline: Technical, Volume Surge, FVG, Momentum, and the two “proprietary” strategies LaunchPad and Alpha Zone. The first four are computed inline inside `_compute_symbol_local()`. LaunchPad and Alpha Zone are computed in a **separate second pass** right after, using the Strategy Engine (`engines/strategies/`) — this is the newer, cleaner code path and the one this document focuses on.

2. What actually starts a scan

There are exactly three ways a scan gets kicked off:

#	Trigger	Where
A	Cron job at 08:00 IST (pre-market check) and 09:00 IST (daily EOD scan)	setup_scheduler() in schedulers/daily_refresh.py

B	Manual trigger from an admin/API call: POST /api/v2/scanner/trigger-scan	api/screener.py → runs run_daily_scan() as a Fa
C	“Lazy build” — if a GET request finds an EMPTY cache collection, it triggers just that one strategy’s pipeline spot-checks the ter	sig @ 57 /api/scanner/spot-checks the ter

Important nuance: trigger A and B run the FULL pipeline (all six scanners, ~4,200 symbols, re-downloads data). Trigger C only rebuilds the ONE strategy whose cache was empty, using whatever OHLCV is already sitting in memory at that moment — it does NOT re-download fresh candles.

3. Step 1 — the data pipeline (run_daily_scan)

This is the full picture from “scan starts” to “UI shows a card.” Read it top to bottom.

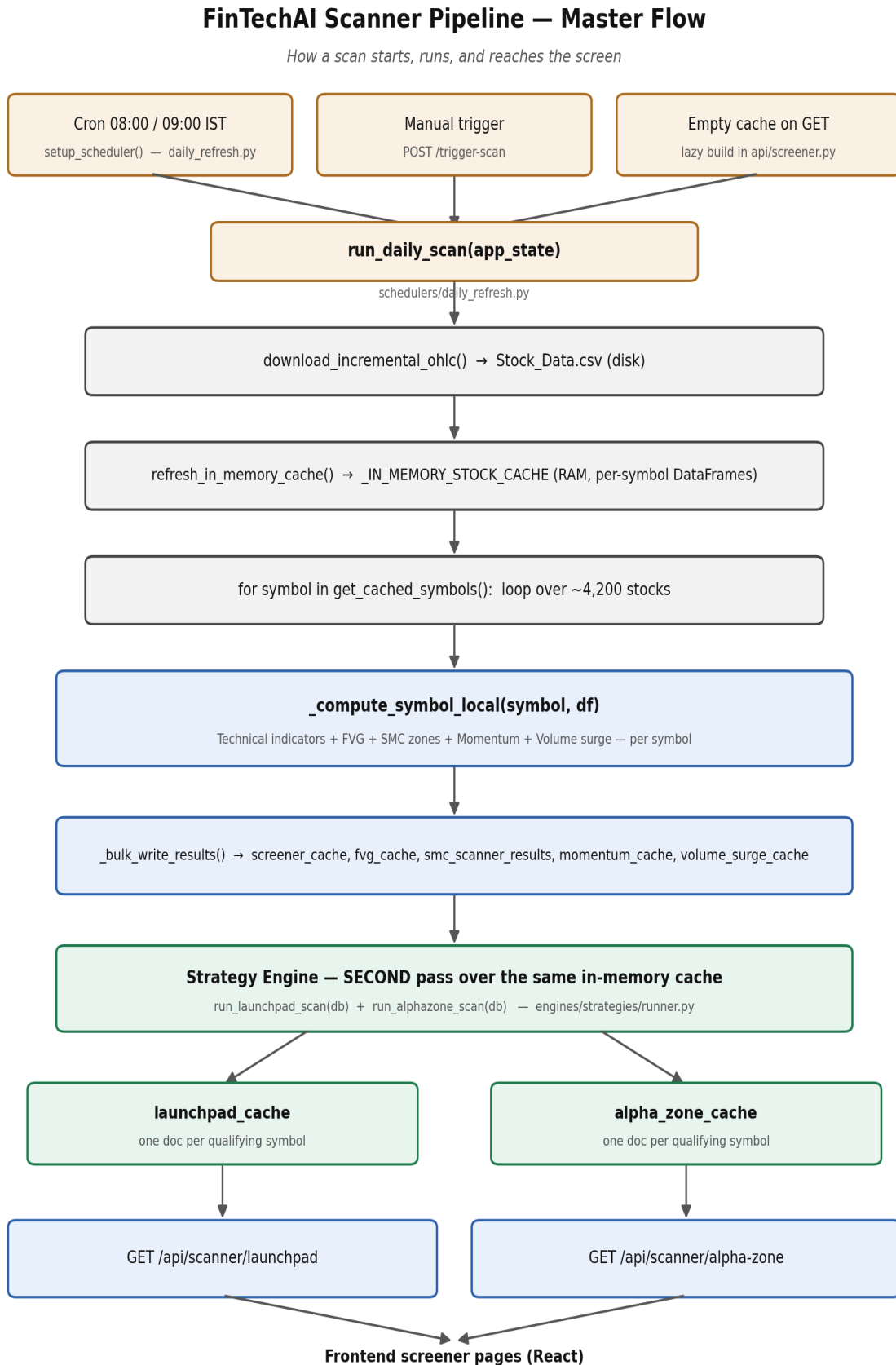


Figure 1 — the master pipeline. Amber = what triggers a scan. Blue = the shared per-symbol computation (technical/FVG/SMC/momentum). Green = the Strategy Engine's second pass that produces LaunchPad and Alpha Zone specifically.

Walking through it in words:

- 1 **run_daily_scan(app_state)** in `schedulers/daily_refresh.py` is the top-level entry point for a full scan.
- 2 It calls `download_incremental_ohlc()` to update `Stock_Data.csv` on disk with the latest candles.
- 3 `refresh_in_memory_cache()` (in `services/ohlc_downloader.py`) reads that CSV once and builds a Python dict `_IN_MEMORY_STOCK_CACHE: {symbol: DataFrame}`. Every scanner reads from THIS dict, not the disk, so scanning is fast.
- 4 The code loops for `symbol in get_cached_symbols()` — one iteration per stock (~4,200 of them).
- 5 For each symbol, `_compute_symbol_local(symbol, df)` computes everything for the OLD-style scanners in one shot: EMA/RSI/MACD indicators, FVG gaps, SMC order blocks, Momentum score, Volume surge stats.
- 6 Results are batched and written with `_bulk_write_results()` into their own Mongo collections: `screeener_cache`, `fvg_cache`, `smc_scanner_results`, `momentum_cache`, `volume_surge_cache`.
- 7 **Only after all of that finishes**, the code calls `run_launchpad_scan(db)` and `run_alphazone_scan(db)` from `engines/strategies/runner.py` — a SEPARATE loop over the same in-memory symbols, using the newer Strategy Engine. This is intentional: LaunchPad and Alpha Zone were rebuilt to be independent of the old FVG/SMC caches and read raw OHLCV directly.
- 8 Each writes its own qualifying rows into `launchpad_cache` / `alpha_zone_cache` using a full `delete_many({})` then `insert_many()` — i.e. every scan REPLACES the whole collection, it doesn't patch individual rows.

4. Step 2 — inside ONE strategy: LaunchPad

LaunchPad looks for a stock in a healthy uptrend where price is sitting inside (or just above) a still-open “Fair Value Gap” — a 3-candle imbalance that tends to act as support. Here is exactly what happens for ONE symbol, called from inside `run_launchpad_scan()`:

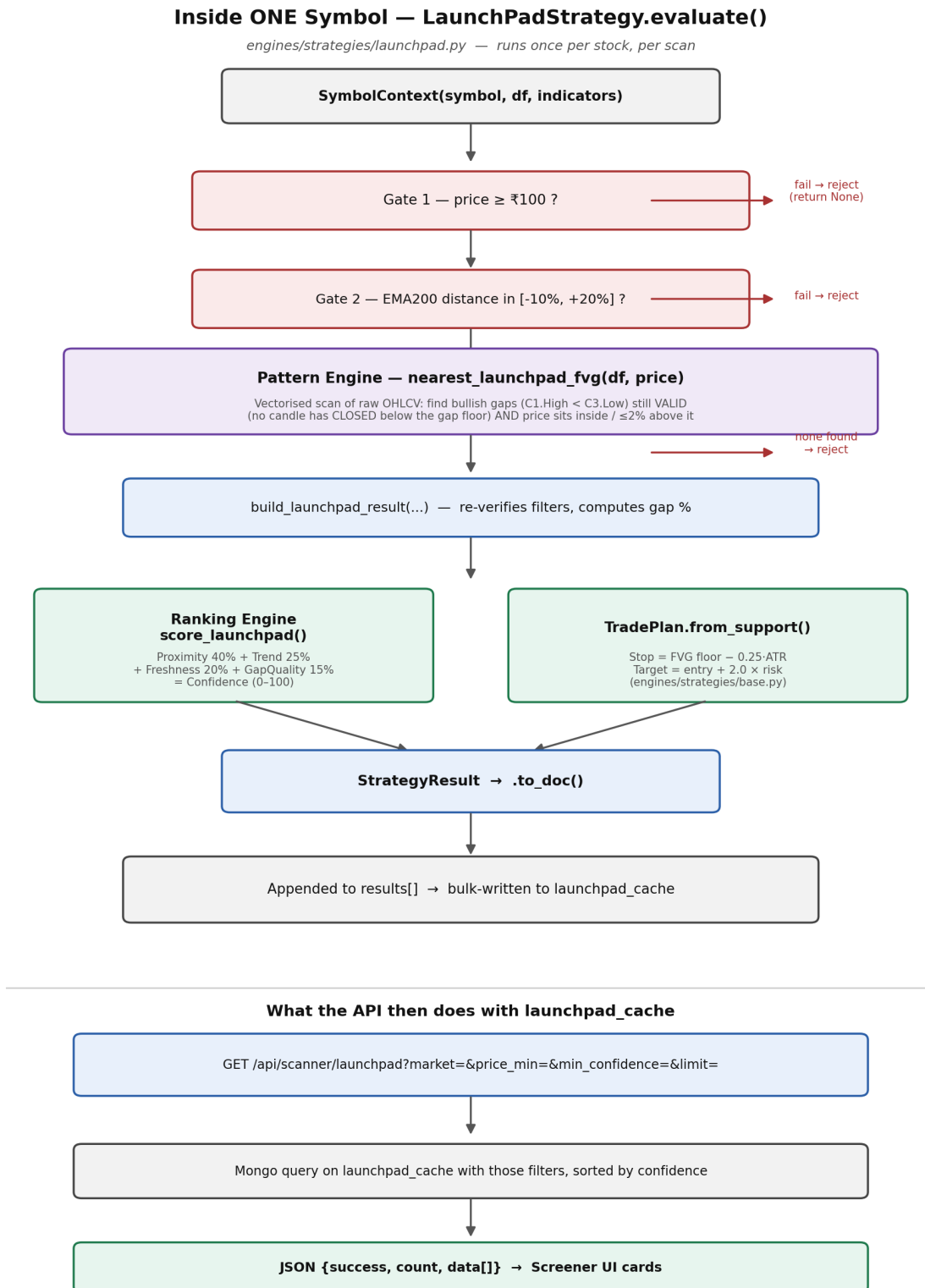


Figure 2 — LaunchPadStrategy.evaluate(), gate by gate, function by function.

In words, the exact call chain:

- 1 `run_launchpad_scan()` builds a `SymbolContext` for the symbol: the raw price `DataFrame` + precomputed indicators (from `IndicatorEngine.compute(df)`).
- 2 `LaunchPadStrategy.evaluate(ctx)` runs two CHEAP gates first so 95%+ of stocks are rejected before any heavier work: (1) price must be $\geq \blacksquare 100$, (2) `EMA200` distance must be between -10% and $+20\%$.
- 3 If both pass, it calls the Pattern Engine: `nearest_launchpad_fvg(df, price)` in `engines/patterns/fvg.py`. This scans the raw candles with numpy (vectorised, not a slow row-by-row loop) to find bullish gaps where `Candle1.High < Candle3.Low`, that are still “valid” — meaning no candle has since CLOSED below the gap floor (a brief wick below is allowed) — AND where price is currently inside the gap or up to 2% above its top.
- 4 If no such gap exists, the symbol is rejected (returns `None`) and the loop moves to the next stock.
- 5 If one is found, `build_launchpad_result(...)` re-validates everything (defensive double-check) and computes the exact gap size, then calls two more engines: `score_launchpad()` (Ranking Engine) for the 0–100 confidence score, and `TradePlan.from_support()` for the entry/stop-loss/target numbers, derived from ATR — nothing hardcoded.
- 6 The result is packaged into a `StrategyResult`, converted to a plain dict with `.to_doc()`, and appended to the results list for that scan.
- 7 After ALL symbols are processed, the whole list is bulk-inserted into `launchpad_cache` in one shot.

5. Step 3 — Alpha Zone (what's different)

Alpha Zone follows the exact same shape (gates → pattern detection → scoring → trade plan → cache) but looks for a different pattern: an **Internal Bullish Order Block** — a zone where institutions likely accumulated — instead of a Fair Value Gap.

Step	LaunchPad	Alpha Zone
Price gate	≥ ■100	≥ ■200
Trend gate	EMA200 dist in [-10%, +20%]	EMA200 dist in [-35%, +40%]
Pattern function	nearest_launchpad_fvg() engines/patterns/fvg.py	nearest_reacting_ob() engines/strategies/alpha_zone_ob.py
Pattern logic	3-candle Fair Value Gap, valid until CLOSE below the floor	Swing-structure Order Block (BOS/CHoCH break), valid until CLOSE below the zone floor
Zone position rule	Inside gap, or ≤2% above top	In Zone / ≤1.5% above / ≤3% "Near Zone"
Scoring function	score_launchpad() — 4 sub-scores	score_confidence() — Institutional + Proximity + Freshness + Trend
Reward multiple	2.0× risk	3.0× risk (longer hold)
Holding period	5–7 days	2–8 weeks
Cache collection	launchpad_cache	alpha_zone_cache

The key architectural point: BOTH strategies read raw OHLCV directly and are fully independent of each other and of the older `scanners/smc_scanner.py` / `scanners/fvg.py` code paths. Neither one calls the other, and neither depends on `fvg_scan_results` or `smc_scanner_results`.

6. Step 4 — how the frontend gets the results

The React screener pages never talk to the engines directly — they only call the REST endpoints in `api/screener.py`:

- 1 **GET /api/scanner/launchpad** and **GET /api/scanner/alpha-zone** accept query params: `market`, `price_min`, `price_max`, `min_return`, `min_confidence`, `limit` (Alpha Zone also has `freshness`, `distance`, `holding_period`).
- 2 Each endpoint FIRST checks if its cache collection is empty (`count_documents({}) == 0`). If so, it runs that one strategy's scan right there (the "lazy build" from Trigger C above) before answering.
- 3 It then builds a Mongo filter dict from the query params, runs `.find(query).sort(...).to_list(length=limit)`, and returns `{success, count, data: [...]}`.
- 4 The frontend page (e.g. `app/screener/launchpad/page.tsx`) fetches this on load, renders one card per result, and shows the `confidence_breakdown` sub-scores in an explainability panel.

7. Confidence & scoring — the exact formulas

LaunchPad confidence (engines/ranking/scorer.py):

$Confidence = 0.40 \times Proximity + 0.25 \times Trend + 0.20 \times Freshness + 0.15 \times GapQuality$

Sub-score	What it rewards
Proximity (40%)	Price near the TOP of the gap (defending support) scores highest; decays smoothly as price stretches above the gap.
Trend (25%)	A healthy uptrend — peaks around +5–10% above EMA200; penalises both a weak/below-EMA trend and an over-ex-
Freshness (20%)	Younger gaps score higher — 100 at day 0, decaying to ~20 by 30 days old.
GapQuality (15%)	Bigger displacement (wider gap %) = stronger imbalance, with diminishing returns above ~2%.

Alpha Zone confidence (engines/strategies/alpha_zone.py):

$Confidence = 0.40 \times Institutional + 0.25 \times Proximity + 0.20 \times Freshness + 0.15 \times Trend$

The “Institutional” sub-score itself is a separate weighted formula computed at the ORDER BLOCK’s formation time (not at scan time) inside alpha_zone_ob.py: 30% displacement off the zone, 25% volume expansion vs. the prior 20 candles, 20% departure candle spread, 15% base tightness before the move, 10% origin-candle absorption wick. Neither strategy uses RSI anywhere — it was deliberately removed from both scoring models.

8. Where things can go quietly wrong (diagnostic map)

If a scanner is showing stale, empty, or unexpected results, these are the exact places in the flow above where a silent failure tends to hide — in the order you should check them.

#	Symptom	Likely cause — check here
1	Cache is empty / stuck on old data	The in-memory OHLC cache (<code>_IN_MEMORY_STOCK_CACHE</code>) only refreshes when <code>refresh_in_memory_cache()</code> is called
2	A symbol you expect is missing from results	Too many silent per-symbol filters run before any strategy logic: <code>MIN_ROWS = 210</code> (needs 210+ candles for EMA200)
3	Scan seems to “hang” or times out from API	<code>Lazy build (Trigger C)</code> runs SYNCHRONOUSLY inside the GET request handler — if the cache is empty will take a while
4	Results look identical to yesterday	Both <code>run_launchpad_scan()</code> and <code>run_alphazone_scan()</code> do a full <code>delete_many({}) + insert_many()</code> — there is no incremental update
5	Confidence / scores look off for a specific stock	Confusion: which gate rejected or passed it by re-deriving by hand: price, EMA200 distance, and (for LaunchPad) <code>score</code>
6	Numbers differ between two scans of the same day	No “same-day” — the only way a change can only enter through the underlying OHLCV changing between runs (e.g. a fresh download)

Fastest starting point: `GET /api/v2/scanner/scan-status` returns the last scan's timestamp, symbols processed/errored, and duration from the `scan_meta` collection — that single call tells you whether the pipeline even ran recently before you dig into any specific stock.